EX: 11, Problem 1: 2-Vertex-Connected Graphs

Problem Statement

Let $G = (V, E)$ be an undirected, unweighted, connected graph without self-loops. The graph G is called *2-vertex-connected* if the graph remains connected whenever an arbitrary vertex is deleted. Formally, G is 2-vertex-connected if for each vertex $u \in V$, the induced subgraph

$$G_u = (V \setminus \{u\}, \{e \in E \mid u \notin e\})$$

is also connected.

Design an algorithm that decides whether G is 2-vertex-connected in $\mathcal{O}(|V| \cdot (|V| + |E|))$ time.

a) General Idea

The graph G is 2-vertex-connected if removing any single vertex does not disconnect the graph. Since G is connected, we test this property by removing each vertex one at a time and checking whether the remaining graph is still connected.

For each vertex $u \in V$, we remove u and all incident edges, and perform a graph traversal (BFS or DFS) on the remaining graph. If for any vertex u the graph becomes disconnected, then G is not 2-vertex-connected. Otherwise, if the graph remains connected for all vertices, G is 2-vertex-connected.

b) Pseudocode

Algorithm 1 Is2VertexConnected

Require: Graph $G = (V, E)$
Ensure: **True** if G is 2-vertex-connected, otherwise **False**

```
if |V| ≤ 2 then return False
end if
for each vertex u ∈ V do
    Choose a vertex s ∈ V \ {u}
    Mark all vertices as unvisited
    Run DFS or BFS from s, ignoring vertex u
    for each vertex v ∈ V \ {u} do
        if v is unvisited then return False
        end if
    end for
end for
    return True
```

c) Running Time Analysis

The outer loop iterates over all $|V|$ vertices. In each iteration, a DFS or BFS is executed, which takes $\mathcal{O}(|V| + |E|)$ time.

Therefore, the total running time is:

$$|V| \cdot (|V| + |E|) = \mathcal{O}(|V| \cdot (|V| + |E|)).$$

d) Correctness Argument

Prove correctness by showing both directions.

If the algorithm returns True: For every vertex $u \in V$, the traversal visits all remaining vertices after removing u. Hence, G_u is connected for all u. By definition, G is 2-vertex-connected.

If G is 2-vertex-connected: By definition, removing any vertex u leaves the graph connected. Therefore, the traversal starting from any remaining vertex reaches all other vertices, and the algorithm never returns False. Hence, it returns True.

Since both directions hold, the algorithm is correct.

e) Python Implementation

```
def is_2_vertex_connected(graph):
    """
    Decide whether an undirected, connected graph
    is 2-vertex-connected.
    """

    vertices = list(graph.keys())

    if len(vertices) <= 2:
        return False

    def dfs(start, removed):
        visited = set()
        stack = [start]

        while stack:
            v = stack.pop()
            if v not in visited:
                visited.add(v)
                for nbr in graph[v]:
                    if nbr != removed and nbr not in visited:
                        stack.append(nbr)
        return visited

    for u in vertices:
        start = next(v for v in vertices if v != u)
        visited = dfs(start, u)
```

```
    for v in vertices:
        if v != u and v not in visited:
            return False

return True
```